Studio Nx

JS 04 — loginUrl()

2026-10-06

JS 04 — `loginUrl(next)`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS · PRD 01 FR-105
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	03 `safeNextPath()` · 05 `parseLoginHash()`

1. 하는 일

로그인 모달로 가는 주소를 만든다. 돌아올 경로가 있으면 `next` 로 붙인다.

`loginUrl('/myPage.html')` → `home.html#login?next=%2FmyPage.html`

2. 언제 쓰나

로그인 화면은 따로 없고 **`home.html` 의 모달**이다 (2026-10-02 결정). 그래서 "로그인하러 가기"는 항상 이 주소로 간다.

상황	부르는 곳
로그인이 필요한 페이지에 로그인 없이 들어옴	`requireLogin()` (07) — `loginUrl(location.pathname + location.search)`
회원가입을 마친 뒤 로그인하러	`join.js` (P1-07) — `loginUrl()`

주소의 `#` 뒤 (해시)

부분	쓰임
`#login`	`home.js` 가 보고 로그인 모달을 자동으로 연다
`?next=…`	로그인 성공 후 돌아갈 경로. `#` 뒤라서 **서버로 보내지지 않고** 브라우저 안에서만 쓰임

보안 — 주소에 `next` 가 보여도 괜찮은가?

한 줄 요약: 주소에 붙는 건 "어디로 가려 했는지"(쪽지)뿐이고, "열쇠"(토큰 · 비밀번호)는 절대 주소에 붙지 않는다.

비유 — 건물 출입

건물	Studio Nx
3층 사무실에 가려는데 출입증이 없다	로그인 없이 `/myPage.html` 에 들어옴
안내 데스크로 보내며 "3층 가려던 분" **쪽지**를 줌	`home.html#login?next=/myPage.html` 로 보냄
안내 데스크에서 신분 확인 → **출입증**을 받음	로그인 → 토큰을 받음
쪽지를 보고 3층으로 안내	`next` 를 읽고 `/myPage.html` 로 이동

쪽지(`next`)는 누가 봐도 괜찮다 — 가려던 층만 적혀 있다. 출입증(토큰)은 주머니(메모리 · HttpOnly 쿠키)에만 넣고 쪽지에는 절대 안 적는다.

순서와 주소창

단계	주소창
① 로그인 없이 마이페이지에 들어옴	`/myPage.html`
② 로그인하러 보냄 — **이때** `next` 가 붙음 (로그인 전)	`home.html#login?next=%2FmyPage.html`
③ 모달에 이메일 · 비밀번호 입력 → 성공	(그대로)
④ `next` 를 검사하고 원래 페이지로	`/myPage.html` — `next` 는 사라짐

걱정과 답

걱정	답
주소에 비밀 정보가 남나?	아니다. 남는 건 경로뿐. 토큰은 메모리 · 쿠키에만
비밀번호가 주소에 붙나?	아니다. 로그인은 `POST` 본문으로 보냄 (주소가 아니라 봉투 안)
서버나 다른 사이트에 새나?	아니다. `#` 뒤는 브라우저 안에만 있고 서버로 가지 않음
누가 `next` 를 가짜 사이트로 바꾸면?	`safeNextPath` 가 우리 사이트 경로만 통과시킴 (03)
`next` 에 스크립트를 넣으면?	`javascript:` 같은 건 거부, 그리고 `next` 는 **이동에만** 쓰고 화면에 HTML 로 넣지 않음
`projectId=3` 이 보이면?	번호는 비밀이 아님. 남의 프로젝트는 서버가 권한으로 막음 (P1-13)

구현 때 지킬 것 (P1-07 로그인 폼)

1. `next` 는 `location.href = next` 처럼 **이동에만** 쓴다 — `innerHTML` 에 넣지 않는다.
2. 로그인 폼은 JS 로 기본 제출을 막고 `apiFetch` 로 보낸다. 폼에 `method="post"` 가 있어 JS 가 실패해도 비밀번호가 주소에 붙지 않는다.

3. 코드

```
export function loginUrl(next) {
  const path = safeNextPath(next);
  return path ? `${LOGIN_PAGE}#login?next=${encodeURIComponent(path)}` : `${LOGIN_PAGE}#login`;
}
```

문법 풀이

식	뜻
`encodeURIComponent(path)`	주소에서 특별한 뜻이 있는 글자를 `%XX` 로 바꿈. `/` → `%2F`, `?` → `%3F`, `=` → `%3D`, `&` → `%26`
`` `${LOGIN_PAGE}#login?next=${…}` ``	**템플릿 문자열**: `${ }` 안의 값을 끼워 넣은 문자열
`path ? A : B`	안전한 경로가 있으면 A(`next` 붙임), 없으면 B(`#login` 만)

4. 흐름

1. `safeNextPath` 로 경로를 검사한다 (03).

2. 안전하면 인코딩해서 `#login?next=` 뒤에 붙인다.
3. 없거나 안전하지 않으면 `home.html#login` 만.

5. 왜 이렇게

선택	이유
`encodeURIComponent`	경로 안의 `?` · `&` · `=` 가 바깥 `?next=` 의 구분자로 잘못 읽히지 않게. 예: `/myProject.html?q=a&page=2` 의 `&page=2` 가 `next` 밖으로 새지 않음
만들 때도 `safeNextPath`	지금은 우리 코드만 부르지만, 잘못된 값이 들어와도 위험한 주소를 만들지 않음 (읽을 때도 한 번 더 검사)
`LOGIN_PAGE` 상수	로그인 페이지가 바뀌어도 한 곳만 고치면 됨

6. 시험

입력	결과
`/myPage.html`	`home.html#login?next=%2FmyPage.html`
`/studioNx.html?projectId=3`	`…next=%2FstudioNx.html%3FprojectId%3D3`
`/myProject.html?q=a&page=2`	`…next=%2FmyProject.html%3Fq%3Da%26page%3D2` — 되읽으면 원래 경로 그대로
없음 · `''`	`home.html#login`
`https://evil.com` · `//evil.com`	`home.html#login` (`next` 안 붙임)